

TECHNOLOGY STACK DOCUMENT

AI Data Conversion Studio



AI-Assisted Source-to-Target Data Migration Mapping Platform
for Insurance / Guidewire modernization programs

Document	Technology Stack & Architecture Reference
Prepared for	Deployment Manager / Release & Operations
Prepared by	Solution Architecture / Rakesh Sinha
Audience	Deployment, Infrastructure & Security reviewers
Version	1.0
Date	23 September 2026
Status	For deployment review
Classification	Confidential — Internal Use

1. Executive Summary

AI Data Conversion Studio helps teams map data from old source systems to new **Guidewire** targets (ClaimCenter, PolicyCenter, BillingCenter) during an insurance data migration. It reads the source system's structure and uses **Claude** (an AI model) to suggest the field-to-field mappings, join logic, value mappings and validation/ETL SQL — which the migration team then reviews and refines.

The design is deliberately simple: a plain **HTML/CSS/JavaScript** website served by one small **Python (Flask)** service on a single address — no separate web server and no build step. The backend keeps no long-running state: it opens short database connections when needed, calls the Claude API through the corporate gateway, and stores users and each client's working data in a local **SQLite** file. It runs as a normal service with `python main.py` (or behind a production WSGI server). The rest of this document lists each part of the stack, its version, and how it is installed.

2. Stack at a Glance

The platform is organized into the following technology layers.

Layer	Core technologies	Role
Presentation	HTML5, CSS3, vanilla JavaScript (ES6, no framework); Bootstrap 5.3, Bootstrap Icons, Chart.js, SheetJS	Browser UI, dashboards, editors
Application / API	Python 3.12, Flask 3 (app-factory + blueprints), Werkzeug	REST-style <code>/api/*</code> over a single origin
AI / GenAI	Anthropic Claude SDK via corporate Bedrock-style gateway (httpx)	Mapping generation, extraction, chat, SQL
Retrieval (RAG)	fastembed (BGE-small ONNX) / NumPy hashing fallback; SQLite-native vector store	Know Your Data document Q&A
Document parsing	openpyxl, pypdf, python-docx, pandas, sqlparse; custom SQL-DDL parser	Excel / PDF / Word / CSV / SQL ingestion
Data connectivity	pyodbc + Microsoft ODBC Driver for SQL Server 17/18	Live source & target metadata, profiling, deploy
Persistence	SQLite (stdlib sqlite3); browser localStorage; in-memory job store	Identity, tenant data, telemetry, working state
Security	Flask signed sessions, Werkzeug scrypt hashing, CSRF, multi-tenant scoping	Auth, isolation, transport trust
Quality	pytest (41 test modules)	Backend regression tests

3. Frontend / Presentation Layer

The frontend is a set of standalone HTML pages under `pages/`, each paired with its own vanilla-JS controller under `js/`. A shared shell (`js/common.js`) injects the sidebar and header, applies the persisted theme, and holds storage/connection helpers. There is **no framework and no build step** — files are served as-is by Flask.

Technology	Version	Purpose
HTML5 / CSS3	—	Static markup; token-based theming with light/dark modes
JavaScript (ES6)	—	Vanilla, framework-free; string-built HTML render functions
Bootstrap	5.3.3	Layout, components, responsive grid (jsDelivr CDN)
Bootstrap Icons	1.11.3	Icon set (jsDelivr CDN)
Chart.js	4.4.1	Dashboard / usage-report charts
SheetJS (xlsx)	0.18.5	Client-side Excel export/read

State model: working data (generated mappings, source/target connections, history, settings) lives in the browser under `aims_*` `localStorage` keys and is mirrored server-side per client. The ~5 MB `localStorage` quota is the main client-side scaling limit for very large schemas.

4. Backend / Application Layer

The backend is a Flask application factory (`create_app()`) launched by `server/main.py`, serving both the static site and the `/api/*` endpoints on `127.0.0.1:8000`. It follows a strict layered architecture with a one-way import direction `api` → `services` → `parsers/schemas` → `core`.

Component	Version	Purpose
Python	3.12	Runtime language (developed/tested on 3.12.5)
Flask	≥ 3.0	Web framework: app factory, 12 blueprints, before/after-request guards
Werkzeug	(with Flask)	WSGI utilities + password hashing (scrypt)
httpx	≥ 0.25, < 1	HTTP client for the Claude gateway (custom CA trust)
NumPy	≥ 1.24	Vector maths for embeddings / similarity
Blueprints	—	static, auth, client, state, db, ai, deploy, ai_usage, admin, kyd, feedback, lookup

Backend package layers

- **core/** — environment/path config, model id resolution, CA-bundle lookup, and centralized optional-import capability guards.
- **parsers/** — pure text/file parsing & chunking (SQL DDL, Excel dictionaries, text/table splitting); no Flask or AI dependency.
- **schemas/** — JSON schemas for Claude structured output.
- **services/** — business logic (auth, clients, DB, AI client, mapping, extraction, KYD/RAG, lookup, deployment, admin, usage); each returns (payload, `http_status`).
- **db/** — multi-tenant SQLite app store (connection, schema, models).
- **api/** — thin blueprints: parse request → call service → jsonify.

5. AI / GenAI Layer

AI features call **Anthropic Claude** through the official Python SDK, routed to a corporate **Bedrock-style gateway** (`ANTHROPIC_BASE_URL + ANTHROPIC_AUTH_TOKEN / ANTHROPIC_API_KEY`). The default model is `bedrock.anthropic.claude-opus-4-8` (Claude Opus 4.8), overridable via `AIMS_MODEL`; the client trusts the corporate TLS-intercepting proxy via a local CA bundle.

Aspect	Detail	Notes
anthropic (SDK)	≥ 0.40	Claude API client for all GenAI calls
Default model	Claude Opus 4.8	<code>bedrock.anthropic.claude-opus-4-8</code> (gateway id)
Transport	httpx + corporate CA bundle	TLS trust for the internal gateway
Structured output	JSON-schema, 4-step ladder	Degrades schema+effort → schema → effort → bare
Streaming	NDJSON progress events	File-extraction progress bar with non-stream fallback
Usage logging	SQLite (<code>aims_usage.db</code>)	Per-call token counts + metadata (telemetry)

How the AI is kept reliable

- **Work in small pieces** — big jobs are split (per target table, and per column for wide tables) so the AI's answer is never cut off; the pieces are merged back together.
- **Stay grounded** — the AI may only use tables and columns that actually exist, so it cannot invent them.
- **Skip the AI when possible** — SQL scripts and well-structured Excel are read directly (no AI — faster and exact); only messy files go to the model.
- **Fail safely** — calls retry and fall back automatically; a failed deployment step is fixed by the AI but always shown for a human to approve, never run on its own.

6. Retrieval-Augmented Generation (Know Your Data)

The **Know Your Data** feature lets users upload insurance documents and chat over them. It runs a full RAG pipeline — ingest → chunk → embed → retrieve → grounded answer — plus text-to-SQL over structured uploads. A query router chooses a vector, structured, or hybrid strategy per question, and answers cite their sources.

Element	Technology	Notes
Embeddings (primary)	fastembed — BAAI/bge-small-en-v1.5	384-dim ONNX semantic vectors, no API key
Embeddings (fallback)	NumPy hashing embedder	256-dim lexical vectors, always available offline
Vector store	SQLite-native (float32 BLOB)	Tenant-scoped cosine similarity in NumPy
Chunking / ingestion	Custom chunker + loaders	PDF/text chunking; CSV/Excel/SQL structured loading
Retrieval scope	<code>user_id + client_id</code>	Per-tenant isolation; optional document scoping

The vector store reuses the existing SQLite database rather than introducing pgvector or a dedicated vector DB — appropriate for the small per-tenant corpora this tool handles. A `sqlite-vec` ANN index can slot in behind the same interface if corpora grow.

7. Document Parsing & Data Connectivity

File ingestion and live-database access are provided by optional libraries guarded by centralized capability checks: if a package is absent the app still boots and reports the missing capability only when a matching request arrives.

Library / Driver	Version	Purpose
openpyxl	≥ 3.1	Excel (.xlsx/.xlsm/.xls) parsing & dictionary extraction
pypdf	≥ 4.0	PDF text extraction
python-docx	≥ 1.1	Word (.docx) parsing
pandas	≥ 2.0	Tabular parsing & data profiling (Know Your Data)
sqlparse	≥ 0.4	Pretty-printing generated SQL (optional, guarded)
pyodbc	≥ 5.1	Live SQL Server metadata, profiling & deployment
MS ODBC Driver	17 or 18	System prerequisite for pyodbc connections

8. Data Storage & Persistence

There is no external database server for application state. Persistence is split across three tiers:

Store	Location / key	Contents
SQLite — app store	server/aims_app.db	Users, clients, per-client tenant documents, KYD documents / chunks / embeddings / chat, lookup value mappings, and Guidewire (CC/PC/BC) dictionary indexes
SQLite — usage	server/aims_usage.db	AI usage telemetry: per-call token counts + metadata
Browser localStorage	aims_* keys	Primary client working state; ~5 MB quota per origin
In-memory	process dict + lock	Background deployment job state (not persisted; history kept client-side)
SQL Server	external, per connection	Source & target customer databases — read live, never copied server-side

Access uses the `sqlite3` module (no ORM) with a process-wide write lock, foreign keys enabled per connection, and short-lived per-operation connections. Every tenant table is scoped by `user_id + client_id` with `ON DELETE CASCADE`.

9. Security & Identity

The application is closed and multi-tenant. Every page and API is gated behind an authenticated session; users are created by an administrator (self-signup disabled by default).

Control	Implementation
Authentication	Flask signed-cookie sessions; HttpOnly + SameSite=Lax; 12 h default lifetime
Password storage	Werkzeug script hashing; plaintext never stored or returned
Brute-force defense	In-memory per-email login throttling with temporary logout
CSRF	Double-submit token on state-changing /api/* requests (defense-in-depth over SameSite)
Multi-tenancy	All tenant data scoped by user_id + client_id; cross-tenant reads prevented by query scoping
Admin / lifecycle	Env-seeded admin, forced-password-change flow, orphan-ingest reconciliation on startup
Secrets handling	Credentials come from env vars / per-request payloads; never written to jobs, logs or prompts
Transport trust	Corporate CA bundle (win-ca-bundle.pem) wired into httpx for the TLS-intercepting proxy

Deployment note: AIMS_SECRET_KEY must be set to a long random value in any real deployment — without it a random per-process key is used and all sessions are invalidated on restart.

10. Deployment

The application is deployed as a single Python/Flask service from the codebase, serving the static site and the API on one origin.

- `pip install -r server/requirements.txt`, then `cd server && python main.py` — serves the whole app (static site + /api/*) at `http://127.0.0.1:8000`.
- A module-level app is exposed, so the same code runs behind a production WSGI server (e.g. waitress / gunicorn) fronted by a reverse proxy.
- The dependency-free `.env` loader reads `server/.env` at startup; real environment variables always win.

11. Installation at a Glance — What, How & When

Every software component, with how and when it is installed. Items marked **Manual** are installed once by hand before the first start; items marked **Automatic** are created by the app itself when it starts — it never installs software on its own.

Software	How it is installed	When
Python 3.12 (runtime)	python.org installer, or OS package manager, on the host	Manual · before start
SQLite (database engine)	Built into Python (sqlite3) — nothing to install	Comes with Python
Flask (+ Werkzeug)	<code>pip install -r server/requirements.txt</code>	Manual · before start
pyodbc	pip (in requirements.txt)	Manual · before start
anthropic SDK (+ httpx)	pip (in requirements.txt)	Manual · before start
openpyxl	pip (in requirements.txt)	Manual · before start
pypdf	pip (in requirements.txt)	Manual · before start
python-docx	pip (in requirements.txt)	Manual · before start
pandas (+ NumPy)	pip (in requirements.txt)	Manual · before start
sqlparse	pip (in requirements.txt)	Manual · before start
pytest (tests only)	pip (in requirements.txt)	Manual · before start
MS ODBC Driver 17/18	Microsoft MSI (msodbcsql18.msi) or winget	Manual · before start
Corporate CA bundle	Generate win-ca-bundle.pem, place in server/	Manual · before start
Configuration / secrets	Set env vars or fill server/.env	Manual · before start
fastembed (better AI search)	<code>pip install fastembed</code> (else NumPy fallback — no install)	Optional · on demand
Bootstrap, Icons, Chart.js, SheetJS	Loaded from the CDN by the browser (or self-host under assets/)	Runtime · in browser
SQLite database files + tables	Created by the app if missing (aims_app.db, aims_usage.db)	Automatic · on start
Seed admin account	Created by the app from AIMS_ADMIN_EMAIL/_PASSWORD	Automatic · on start
fastembed model (ONNX)	Downloaded automatically — only if fastembed is installed	Automatic · first use

In short: install the Manual items once, then start the server with `python main.py` — the app creates its databases, tables and admin user on startup. Nothing is *pip-installed* at start.

12. Runtime Prerequisites & Configuration

Environment / platform

- **Python 3.12+** on the server host.
- **Microsoft ODBC Driver for SQL Server 17 or 18** for live SQL connections.
- **Outbound access** to the corporate Claude gateway and the CDN assets (Bootstrap, Chart.js, SheetJS) — or self-host the CDN assets in a locked-down network.
- **Corporate CA bundle** (server/win-ca-bundle.pem) rebuilt on the host for the TLS-intercepting proxy (gitignored).

Key configuration variables

Variable	Category	Purpose
ANTHROPIC_BASE_URL	AI (required)	Corporate / Bedrock gateway base URL
ANTHROPIC_AUTH_TOKEN / _API_KEY	AI (required)	Gateway credential
AIMS_SECRET_KEY	Auth (required)	Signs session cookies; set a long random value
AIMS_ADMIN_EMAIL / _PASSWORD	Auth	Bootstraps the seed administrator account
AIMS_MODEL	AI (optional)	Override the default Claude model id
PORT	Optional	Dev server port (default 8000)
AIMS_SESSION_HOURS / _CSRF_ENABLED / _SIGNUP_ENABLED	Optional	Session lifetime; CSRF (on); self-signup (off)
AIMS_APP_DB / _USAGE_DB / _KYD_MAX_UPLOAD_MB	Optional	SQLite paths; max KYD upload (25 MB default)

Secrets are injected as environment variables (e.g. via a secret manager) or a gitignored server/.env; no credentials are committed to source control.

13. Quality, Constraints & Scaling Notes

- **Testing:** pytest with 41 backend test modules (routing, auth, admin, AI plumbing, parsers, services). There is no frontend build/lint tooling; frontend verification is manual in the browser.
- **Concurrency:** the dev server runs single-process with SQLite single-writer semantics (a process-wide write lock). For higher concurrency, run behind a WSGI server and consider migrating SQLite to a server database.
- **Client storage:** the ~5 MB browser localStorage quota bounds very large schemas on the client.
- **Graceful degradation:** optional features (SQL Server, Excel/PDF/Word, AI) are capability-guarded — the app boots and serves static content even when a dependency or credential is missing.
- **Statelessness:** the backend holds no long-lived per-request state except the in-memory deployment job store, which is intentionally not persisted.

This document reflects the technology stack as implemented in the current codebase and is intended as the architecture reference for the deployment review.